



Primeiros programas em C

Como um programa nasce, compila e roda — os tipos e operadores que ele manipula, e como ele toma decisões.

`code.c`



`gcc`



`./programa`

O que vamos ver

01 Do código ao executável

como um programa em C nasce, compila e roda

02 Variáveis e tipos

onde os dados moram na memória e quanto espaço ocupam

03 Operadores e E/S

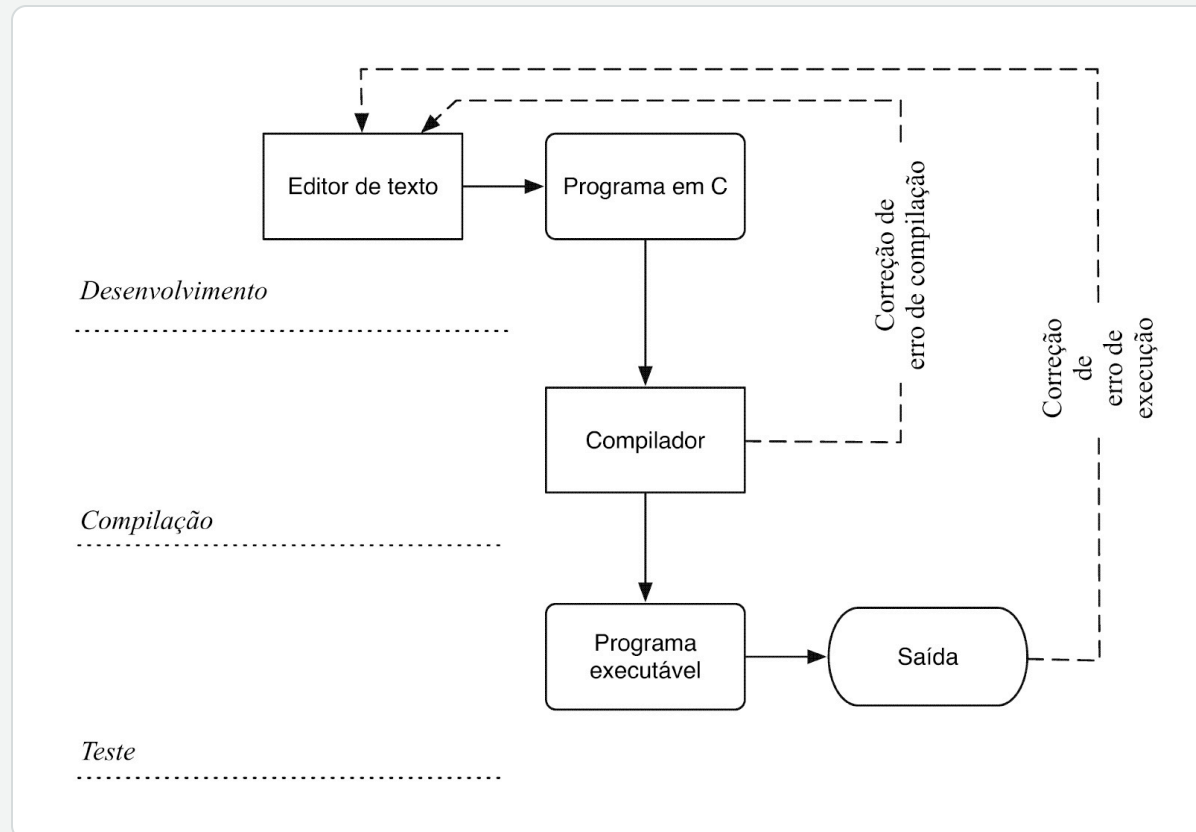
como transformar valores e conversar com quem usa o programa

04 Decisões

como o programa escolhe o que fazer com if e switch

Editar, compilar, testar — em ciclo

Um erro de compilação ou de execução manda de volta para o editor.



A mesma anatomia, sempre

Todo programa em C segue este esqueleto.

#include

traz as declarações de uma biblioteca (ex.: stdio.h)

int main()

ponto de entrada obrigatório — é daqui que a execução começa

{ }

delimitam o corpo: comandos executados em ordem, de cima a baixo

return 0;

encerra a função e devolve um status ao sistema operacional

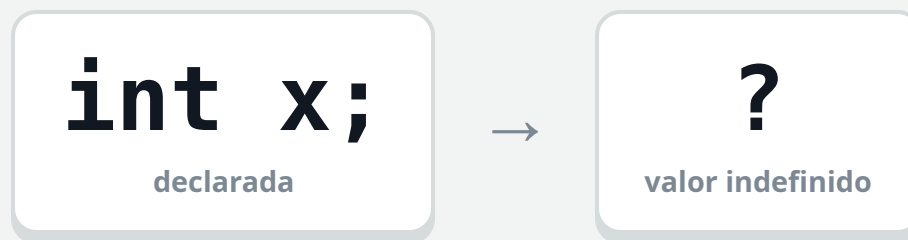
Cada tipo ocupa um número fixo de bytes

char, int, float, double e void — sizeof revela o tamanho de cada um.

char		1 byte
int		4 bytes
float		4 bytes
double		8 bytes

Declarar não é inicializar

Sem atribuição explícita, uma variável recém-declarada guarda "lixo de memória" — e o compilador pode avisar.



```
07_CONVERSION_AND_UNINITIALIZED/MAIN.C
int nao_inicializada;
printf("%d\n", nao_inicializada);
// gcc -Wall avisa:
// warning: 'nao_inicializada'
// is used uninitialized
```

Nem toda divisão sobra decimais

O tipo dos operandos decide se o resultado trunca — o cast explícito força a conversão

7 / 2

Os dois operandos são int — divisão inteira, trunca.

3

7.0 / 2

Um operando é ponto flutuante — preserva as casas decimais.

3.5

```
07_CONVERSION_AND_UNINITIALIZED/MAIN.C
```

```
// os dois sao int: trunca
float media_errada =
    total_pontos / quantidade_provas;
// (float) antes da divisao:
float media_certa =
    (float) total_pontos / quantidade_provas;
```

`++x` e `x++` não são a mesma coisa

A diferença aparece quando o operador está dentro de uma expressão maior.

`x++`

Usa o valor atual — o incremento acontece depois.

**usa 5, x
vira 6**

`++x`

Incrementa primeiro — a expressão já usa o novo valor.

**x vira
7, usa 7**

```
05_INCREMENT_DECREMENT/MAIN.C
```

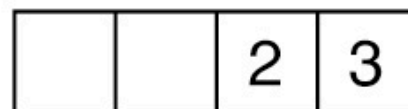
```
int resultado_pos = x++;  
// resultado_pos = 5, x agora = 6
```

```
int resultado_pre = ++x;  
// x agora = 7, resultado_pre = 7
```

printf também controla a largura

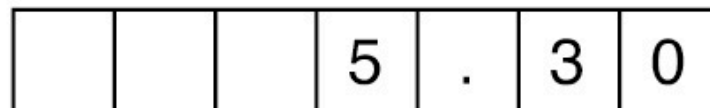
`%4d` e `%7.2f` reservam um número mínimo de posições, alinhadas à direita.

`%4d`



└── 4 ──┘

`%7.2f`



└── 2 ──┘

└── 7 ──┘

scanf sempre pede o endereço

Sem o &, scanf não sabe onde guardar o valor lido.



&&, || e ! combinam condições

E o operador `?`: escolhe um valor sem precisar de if-else.

a && b

verdadeiro
só se os dois
lados forem
verdadeiros

a || b

verdadeiro
se pelo
menos um
lado for

!a

inverte um
valor lógico

06_LOGICAL_AND_TERNARY/MAIN.C

```
int elegivel =  
    maior_de_idade && renda_baixa;  
  
const char *faixa =  
    idade < 18 ? "menor de idade" :  
    (idade < 60 ? "adulto" : "idoso");
```

C não tem um tipo booleano

Qualquer expressão numérica pode ser usada como condição de um if.

0

Tratado como falso — o bloco do if não executa.

```
if (0) → não executa
```

≠ 0

Qualquer valor diferente de zero é tratado como verdadeiro.

```
if (5) → executa
```

switch testa em cascata, até bater

Sem break, a execução "cai" para o case seguinte (fall-through).

```
case '+'
```

```
case '-'
```

```
case '*'
```

```
case '/' ← bate aqui, executa até o break
```

```
default só roda se nenhum case bater
```

AGORA É COM VOCÊ

Explore os exemplos, no seu ritmo

Código-fonte, explicação linha a linha e simulação da execução passo a passo — sem precisar compilar nada. Depois, teste o que aprendeu no quiz.

 **Notas de Aula e Exemplos** thiagopx.github.io/ed/01_intro

 **Quiz de fixação** thiagopx.github.io/ed/01_intro/quiz